

Multi-Agent Bug Detection & Auto PR System

Autonomous Software Maintenance via 10-Agent Orchestrated Pipeline with Knowledge-Graph-Backed Cross-File Reasoning

An autonomous multi-agent system that builds a semantic knowledge graph of a GitHub repository, discovers functional bugs, security vulnerabilities, code quality issues, and performance problems, plans ordered repairs using topological dependency analysis, generates validated patches through a fix-verify feedback loop, and publishes confidence-scored pull requests with human approval gates. The system is delivered as a Streamlit application that drives the agent pipeline directly and streams live agent progress to the user.

Table of Contents

1. Problem Understanding

- 1.1 Four Issue Classes
- 1.2 Why Naive Approaches Fail

2. Agent Architecture Design

- 2.1 Ten-Agent Reference Table
- 2.2 Phase Organisation

3. Knowledge Graph Architecture

- 3.1 Graph Schema
- 3.2 Graph Construction Pipeline
- 3.3 Cross-File Reasoning via Graph Traversal

4. Workflow & Agent Orchestration

- 4.1 Phase 1: Parallel Discovery
- 4.2 Phase 4: Fix-Validate Feedback Loop
- 4.3 Repository Memory (ChromaDB)

5. Cross-File Bug Investigation

- 5.1 Data-Flow Tracing
- 5.2 Blast Radius Analysis
- 5.3 Interactive Bubble Map (Blast-Radius Visualisation)

6. Validation Pipeline

- 6.1 Four-Gate Validation
- 6.2 Differential Security Analysis
- 6.3 Regression Detection Strategy

7. Confidence Scoring & Human Approval

- 7.1 Scoring Signals & Weights
- 7.2 Critical Path Detection
- 7.3 Approval Gate Workflow

8. Technology Stack

9. Data Flow & Integrations

10. Cost Control & Observability

- 10.1 LLM Cost Optimisation
- 10.2 Observability & Tracing

10.3 Error Handling & Resilience

11. Expected Outcomes

12. Scalability & Advanced Features

13. Innovation vs Existing Tools

14. Limitations & Future Work

1. Problem Understanding

Autonomous software maintenance demands more than a scan-and-patch pipeline. It asks for a system that behaves like a senior engineer doing a complete audit: first building *deep understanding* of a repository before touching a single line, then discovering four distinct classes of issues (functional bugs, security vulnerabilities, code quality problems, and performance issues), then investigating root causes rather than symptoms, planning fixes in the right order, generating validated patches that actually pass tests, and finally producing structured pull requests with confidence scores and human approval gates for high-risk changes.

1.1 Four Issue Classes

Table 1 — Issue Classes, Examples, and Primary Detection Methods

Issue Class	Examples	Primary Detection Method
Functional Bugs	Null pointer exceptions, missing edge cases, incorrect conditions, broken APIs	LLM reasoning over file content + knowledge graph context + data-flow tracing
Security Vulnerabilities	SQL injection, XSS, CSRF, hardcoded secrets, weak authentication, JWT issues	LLM Bug Hunter (Haiku triage → Sonnet confirm) + Bandit + differential re-scan
Code Quality	Dead code, duplicate code, long methods, circular dependencies	Static analysis (ESLint, pylint) + LLM pattern reasoning + graph cycle detection
Performance	Memory leaks, inefficient loops, N+1 database queries	LLM reasoning over ORM patterns and loop structures in knowledge graph + data-flow analysis

1.2 Why Naive Approaches Fail

Existing tools — Patchwork, CodeRabbit, Semgrep Autofix — share a common failure: they treat each file in *isolation*. They cannot answer questions that a senior engineer asks instinctively: "*Why does this bug exist?*", "*Which modules are affected?*", or "*What is the safest fix order?*" They also cannot verify that a fix actually *works* — they generate a patch and open a PR with no test execution.

Key Insight

Our system addresses every one of these gaps through a ten-agent orchestrated pipeline with (1) a semantic knowledge graph for cross-file reasoning, (2) a two-stage LLM Bug Hunter (cheap Haiku triage → strong Sonnet confirmation) that finds real defects with cited evidence rather than style nits, (3) surgical anchored-diff patching that changes only what it must (no whole-file rewrites), (4) a real fix-validate feedback loop with differential security verification, and (5) repository memory of past fixes for accelerated learning.

In essence: an autonomous software maintenance agent that builds a knowledge graph of the repository, discovers all four issue classes through cross-file analysis, plans an ordered repair strategy via topological dependency sorting, generates patches that pass actual tests, verifies no new vulnerabilities are introduced, and publishes confidence-scored, human-approved pull requests.

2. Agent Architecture Design

Ten specialised agents organised into five phases, coordinated by an Orchestrator that controls sequencing, decides retries, and handles partial failures. Phase 1 runs in parallel (three agents concurrently). Phase 4 is a feedback loop: Code Gen → AST Validator → Validation Agent → Security Verify, with retry arrows back to Code Gen on any failure.

2.1 Ten-Agent Reference Table

Table 2 — Agent Reference: Responsibilities, Tools, and Design Decisions

Agent	Phase	Core Responsibility	Key Tools
1. Repo Mapper	1 (parallel)	Build knowledge graph: language, framework, modules, API endpoints, DB interactions, service boundaries	ast module, grep, file tree walk, networkx
2. Dependency Analyzer	1 (parallel)	Parse requirements.txt / package.json / pom.xml; detect outdated and CVE-flagged packages	pip-audit, npm audit, safety
3. Bug Hunter	1 (parallel)	Two-stage LLM detection — Haiku triages every source file, Sonnet confirms the real defects with cited evidence and a concrete fix — complemented by local scanners; normalised findings across all four issue classes	Claude Haiku + Claude Sonnet, Bandit, ESLint
4. Bug Investigation	2	For each finding: determine root cause, severity, impact radius, affected modules; query ChromaDB for similar past fixes; perform cross-file data-flow tracing via knowledge graph	Claude Sonnet + 2-hop BFS on knowledge graph + ChromaDB semantic search
5. Repair Planner	3	Order fixes by dependency (fix A before B if A's fix affects B's file); generate repair plan; compute confidence pre-score; prioritise security fixes	Claude Sonnet + topological sort on affected file graph
6. Code Generation	4 (loop)	Generate a minimal patch per epicenter file using anchored SEARCH/REPLACE edits (never a whole-file rewrite); all of a file's bugs fixed in one call with full-file context; emits a real unified diff	Claude Sonnet + anchored-diff patcher
7. Validation Agent	4 (loop)	Apply the patch to a sandbox; run pytest + bandit; honest no-tests fallback (cap confidence at 60%); on failure return error to Code Gen; detect regressions via graph blast radius	pytest, bandit, subprocess, temp copy sandbox
8. Security Verification	4 (loop)	Re-run the security scan to confirm the original vulnerability is gone; perform differential analysis to	Bandit targeted re-scan + differential

Agent	Phase	Core Responsibility	Key Tools
		detect newly introduced vulnerabilities	comparison
9. PR Author	5	Compute final confidence score; apply human approval gate for high-risk changes; open GitHub PR with structured description and blast radius summary	PyGithub, confidence scorer, critical-path detector
10. Orchestrator	All	LLM-based controller: decides what runs next, when to retry, when to escalate, when to mark as unresolvable; adapts to partial failures	Claude (reasoning over pipeline state) + LangGraph StateGraph

2.2 Phase Organisation

Table 3 — Pipeline Phase Summary

Phase	Agents	Execution	Output
Phase 1: Discovery	Repo Mapper, Dependency Analyzer, Bug Hunter	Parallel (async)	Knowledge graph + CVE list + findings
Phase 2: Investigation	Bug Investigation	Sequential per finding	Root causes + blast radius + similar fixes
Phase 3: Planning	Repair Planner	Sequential (depends on Phase 2)	Ordered repair plan + confidence pre-score
Phase 4: Fix-Validate Loop	Code Gen, Validation Agent, Security Verification	Loop with retry (max 3)	Validated patch set + security confirmation
Phase 5: Publication	PR Author	Sequential	GitHub PR with confidence badge + approval gate

3. Knowledge Graph Architecture

The knowledge graph is the *backbone* of the entire system. Unlike tools that analyse files in isolation, our system builds a directed graph of the entire repository to enable cross-file reasoning, blast radius analysis, and dependency-aware repair ordering. This is the critical architectural differentiator.

3.1 Graph Schema

The knowledge graph is stored as a `networkx.DiGraph` with typed nodes and edges. The schema is designed to capture structural, behavioural, and framework-specific relationships.

Table 4 — Knowledge Graph Node Types

Node Type	Attributes	Source
File	path, language, size, last_modified	File tree walk
Function	name, signature, line_start, line_end, decorators, complexity	AST parsing (ast module)
Class	name, bases, methods, line_start, line_end	AST parsing
APIEndpoint	route, method, handler_function, auth_required	Framework-specific regex (FastAPI decorators, Flask routes, Django URL patterns)
DBModel	table_name, fields, relationships	ORM introspection (SQLAlchemy, Django ORM, Prisma)
ServiceBoundary	name, entry_points, external_deps	Directory structure + config files
ConfigVar	key, value (masked), source_file	Environment variable + config file scan
Secret	pattern_matched, confidence, location	Secret-pattern scan (regex + LLM review)

Table 5 — Knowledge Graph Edge Types

Edge Type	Source	Target	Detection Method
IMPORTS	File	File / Module	AST Import / ImportFrom nodes
CALLS	Function	Function	AST call-node resolution + import graph
DEFINES	File	Function / Class	AST top-level definitions
SERVES_API	Function	APIEndpoint	FastAPI: <code>@app.get</code> decorators; Flask: <code>@route</code> ; Django: <code>urls.py</code>

Edge Type	Source	Target	Detection Method
QUERIES_DB	Function	DBModel	ORM method calls: <code>.filter()</code> , <code>.all()</code> , <code>.query()</code>
BELONGS_TO	Function / Class	ServiceBoundary	File path prefix matching
USES_CONFIG	Function / File	ConfigVar	String matching on <code>os.environ</code> / <code>config.get</code>
RETURNS_TO	Function	Function	Data-flow: return value consumed by caller

3.2 Graph Construction Pipeline

The Repo Mapper agent constructs the graph in four passes to handle both static and dynamic patterns:

Pass 1: Structure Discovery. File tree walk creates `File` nodes and directory-based `ServiceBoundary` nodes.

Pass 2: AST Parsing. Parse every Python file with `ast.parse` to create `Function`, `Class`, `IMPORTS`, and `DEFINES` edges. For JavaScript/TypeScript, use Babel parser via `@babel/parser`.

Pass 3: Framework-Specific Extraction. Regex-based pattern matchers extract framework-specific relationships:

- **FastAPI:** Match `@app.get/post/put/delete` decorators to create `APIEndpoint` nodes and `SERVES_API` edges
- **Django:** Parse `urls.py` for route definitions; parse `models.py` for `DBModel` nodes
- **Flask:** Match `@app.route` decorators
- **ORMs:** Match `session.query(Model)`, `Model.objects.filter()`, `db.query(Model)` for `QUERIES_DB` edges

Pass 4: Dynamic Pattern Handling. For patterns AST alone cannot capture (decorators with factory functions, monkey-patching, dynamic imports), conservatively mark edges as `confidence: low`. These edges widen the blast radius rather than being ignored.

Conservative Over-Approximation Principle

When the graph cannot definitively resolve a relationship (e.g., dynamic imports, metaclass factories), the system creates an edge marked with `confidence: low` rather than omitting it. This ensures the blast radius is never underestimated, at the cost of potentially wider (but safe) repair scopes.

3.3 Cross-File Reasoning via Graph Traversal

The Bug Investigation agent uses the knowledge graph to perform *cross-file root cause analysis* that isolated file scanning cannot achieve:

2-Hop BFS Pattern. Starting from the file where a finding was flagged, the agent performs a 2-hop breadth-first search along `IMPORTS` and `CALLS` edges to identify:

- **Callers:** Which functions call the suspect function (upward traversal)
- **Callees:** Which functions does the suspect call (downward traversal)
- **Data sources:** Which `DBModel` or `APIEndpoint` feeds data into the suspect code

Example: Cross-File Null Pointer. Suppose the Bug Hunter flags a potential null dereference in `api.py:process_order()`. The Bug Investigation agent queries the graph:

1. `process_order` has a `QUERIES_DB` edge to `OrderModel`
2. `OrderModel` is defined in `models.py`
3. Reverse `CALLS` edges show `process_order` is called by `create_order` (in `views.py`) and `bulk_import` (in `admin.py`)
4. The null originates from `bulk_import` not validating CSV fields before passing to `process_order`

Result: The root cause is identified as missing input validation in `admin.py`, not a bug in `api.py` where the symptom appeared. The repair plan fixes the source, not just the symptom.

4. Workflow & Agent Orchestration

The Orchestrator Agent uses LLM reasoning over the current pipeline state to decide what runs next — not a hardcoded DAG. This means it can handle partial failures gracefully: if the Dependency Analyzer fails to reach the CVE registry, the Orchestrator skips that finding source and continues with the remaining detectors rather than aborting the entire run.

4.1 Phase 1: Parallel Discovery

Repo Mapper, Dependency Analyzer, and the Bug Hunter all receive the cloned repository path and run concurrently via `asyncio.gather`. None depends on the others. The Bug Hunter's cheap Haiku triage pass scans files in parallel and only the flagged files are escalated to the Sonnet confirmation pass; the knowledge graph build takes 2–3 seconds. Discovery completes before the Orchestrator advances to Phase 2. This parallelism is the biggest single time-saver in the pipeline.

4.2 Phase 4: Fix-Validate Feedback Loop

Every patch set goes through four gates before a PR is opened:

Table 6 — Four-Gate Validation Pipeline

Gate	Check	Tool	Cost	On Failure
Gate 1	AST syntax validation	<code>ast.parse</code>	Zero (local)	Immediate retry with Code Gen
Gate 2	Test execution + static analysis	pytest + bandit	Low (subprocess)	Return error output to Code Gen, retry
Gate 3	Regression detection	pytest on blast-radius modules	Medium	Return regression details to Code Gen
Gate 4	Differential security analysis	Bandit diff scan	Low	Security-specific retry or human escalation

Gate 1 is a zero-cost AST syntax check that catches malformed Python immediately with no API call. Gate 2 applies the patch to a sandboxed copy of the repo, runs pytest for the affected modules, and runs bandit on the patched files. Gate 3 runs tests on modules within the blast radius (identified via the knowledge graph) to detect regressions. Gate 4 performs differential security analysis (see Section 6.2). Only after all four gates pass does the Orchestrator advance to the PR Author. If any gate fails, the Orchestrator returns control to Code Gen with the full error output appended to the prompt, up to three retry attempts.

4.3 Repository Memory (ChromaDB)

Every successfully validated fix is stored in ChromaDB with four fields: the bug type, the root cause pattern, the fix strategy used, and the confidence score achieved. When the Bug Investigation agent encounters a new issue, it queries ChromaDB for semantically similar past fixes using the bug type and affected code snippet as the query vector. A cache hit boosts the initial confidence pre-score and provides the Repair Planner with a proven strategy to start from, reducing both LLM calls and retry attempts.

The memory collection uses `bug_type + root_cause_pattern` as the embedding text and stores the full fix strategy as metadata. Querying uses cosine similarity with a threshold of 0.85; results above this threshold are considered "similar past fixes."

5. Cross-File Bug Investigation

This section details how the Bug Investigation agent performs reasoning that spans multiple files — the defining capability that separates this system from isolated scanners.

5.1 Data-Flow Tracing

When a finding is flagged in file `F`, the Bug Investigation agent performs *data-flow tracing* along the knowledge graph:

1. **Backward trace (taint analysis):** Follow `CALLS` and `RETURNS_TO` edges upstream to find where the suspect data originates. Identify all `APIEndpoint` and `DBModel` nodes that feed into the suspect code path.
2. **Forward trace (impact analysis):** Follow `CALLS` edges downstream to find all functions that depend on the suspect code. This determines the *blast radius*.
3. **Cross-boundary analysis:** If the trace crosses a `ServiceBoundary` edge, flag the fix as higher-risk (requires integration test verification).

Concrete Example: SQL Injection Chain. The Bug Hunter flags `cursor.execute(query)` in `db.py:run_query()`. Data-flow tracing reveals:

- `run_query` is called by `search_products` (in `products.py`) which receives `query` from `GET /api/search?q=...`
- The `q` parameter is never sanitised between the API endpoint and the database call
- Three other endpoints (`/api/filter`, `/api/advanced_search`, `/admin/export`) also call `run_query` with unsanitised input

Result: The root cause is not in `db.py` but in the missing input validation layer. The repair plan adds a sanitisation function in a shared utility module and updates all four endpoints — a fix the original file-level scanner would never discover.

5.2 Blast Radius Analysis

The blast radius of a proposed fix is computed as the set of all nodes reachable within k hops from the modified file in the knowledge graph. By default $k = 2$ (configurable).

Table 7 — Blast Radius Categories and Actions

Radius	Affected Nodes	Required Action
Narrow (0–1 hop)	1–2 files	Run tests on affected files only
Moderate (2 hops)	3–8 files	Run tests on blast-radius modules + regression check

Radius	Affected Nodes	Required Action
Wide (3+ hops)	9+ files or crosses service boundaries	Full test suite + mandatory human approval

5.3 Interactive Bubble Map (Blast-Radius Visualisation)

The blast radius is exposed directly to the user as an **interactive bubble map** of the repository, rendered in the Streamlit UI. After a repository URL is entered, the Repo Mapper builds the knowledge graph (pure AST + regex, no LLM call), and the UI projects it to a file-level graph: every source file becomes a bubble, and two bubbles are connected when the files share a structural relationship (an imported module, a called function, or a common service boundary). Bubble size reflects file size, and bubbles are grouped by service boundary.

The interaction makes cross-file impact tangible: **clicking any bubble turns that file red and recolours every file in its blast radius orange** — i.e., the set of files that would be affected if that file were changed, computed as the file nodes reachable within $k = 2$ hops in the knowledge graph (the same `blast_radius()` query the Validation and PR agents use). All other files fade to grey, so the impact set is immediately legible; clicking empty space resets the view. This gives the engineer a senior-reviewer's mental model of "what does this file touch?" before any change is planned.

Why It Matters

The bubble map turns the otherwise-invisible knowledge graph into a direct manipulation interface. The same graph that drives automated repair ordering and regression scoping is the one the human explores — there is a single source of truth for "connected files" and "blast radius," whether it is consumed by an agent or by a person clicking a bubble.

6. Validation Pipeline

6.1 Four-Gate Validation

(See Table 6 in Section 4.2 for the full gate breakdown.) The Validation Agent applies patches to a temporary copy of the repository to ensure the working tree is never contaminated. It runs pytest with `-x` (fail fast) on affected test modules, captures stdout/stderr for retry context, and runs bandit on changed files only for efficiency.

6.2 Differential Security Analysis

A critical weakness of naive fix-and-scan approaches is that they only verify the *original* vulnerability is gone. LLM-generated code can introduce *new* vulnerabilities — missing input sanitisation, weakened authentication checks, or new injection points.

Our Security Verification agent performs **differential security analysis**:

1. **Baseline capture:** Before applying the patch, run Bandit (and any locally available scanners) on the entire changed file, recording all findings with line numbers.
2. **Post-fix capture:** After applying the patch, re-run the same scans on the same file.
3. **Diff comparison:** Compare the two finding sets:
 - The original vulnerability finding must be *absent* in the post-fix scan
 - No *new* security findings (with severity $\geq$ "medium") may appear in the post-fix scan that were not in the baseline
4. **Flagging:** If new medium+ severity findings appear, the patch is rejected with a security-specific error message, and Code Gen is instructed to fix the introduced issue.

6.3 Regression Detection Strategy

The Validation Agent detects regressions through two mechanisms:

Blast-Radius Testing. Beyond running tests for the directly affected module, the agent queries the knowledge graph for all modules within the blast radius (see Section 5.2) and runs their test suites. A failing test in a downstream module signals a regression.

Fallback for Repositories Without Tests. If the target repository has no test suite (detected by absence of `test_*.py` or `*_test.py` files), the Validation Agent falls back to:

- Static analysis only (bandit + LLM review)
- Maximum confidence score capped at 60% (tests passed is the strongest signal at +40%)
- Automatic human approval gate triggered regardless of other signals

7. Confidence Scoring & Human Approval

7.1 Scoring Signals & Weights

Table 8 — Confidence Score Signals and Weights

Signal	Weight	Rationale
pytest suite passes for affected modules	+40%	Strongest signal: behaviour is preserved
Post-fix bandit + differential scan clean	+25%	Vulnerability is confirmed removed, no new ones introduced
AST syntax valid (Gate 1)	+10%	Structural correctness of generated code
ChromaDB cache hit (similar past fix succeeded)	+15%	Historical precedent improves confidence
Repair Planner ordered this fix first	+10%	Low dependency risk (topological sort priority)

The final score is a weighted sum: **Score** = **40%** × **tests** + **25%** × **security_clean** + **10%** × **ast_valid** + **15%** × **cache_hit** + **10%** × **fix_order**. Each signal is binary (pass/fail), producing a score in the range [0%, 100%].

7.2 Critical Path Detection

A fix is flagged as "critical path" if any of the following conditions are met:

Table 9 — Critical Path Detection Rules

Rule Type	Pattern	Example Matches
File path regex	<code>.*auth.*\.py</code> , <code>.*jwt.*</code> , <code>.*crypto.*</code> , <code>.*password.*</code> , <code>.*login.*</code> , <code>.*session.*</code> , <code>.*oauth.*</code>	<code>auth_service.py</code> , <code>jwt_utils.py</code> , <code>crypto_helpers.py</code>
Function name regex	<code>.*authenticate.*</code> , <code>.*authorize.*</code> , <code>.*hash.*</code> , <code>.*encrypt.*</code> , <code>.*sign.*</code>	<code>authenticate_user()</code> , <code>hash_password()</code>
Graph edge	Modifies a function with <code>SERVES_API</code> edge where <code>auth_required=true</code>	Any protected endpoint handler
Security severity	Original finding severity is "critical" or "high"	SQL injection, authentication bypass

Rule Type	Pattern	Example Matches
Crosses service boundary	Blast radius includes a <code>ServiceBoundary</code> edge	Auth service calling user service

7.3 Approval Gate Workflow

Table 10 — Approval Gate Decision Matrix

Confidence Score	Critical Path?	Action
$\geq 70\%$	No	Open PR as ready-to-merge; notify team via webhook
$\geq 70\%$	Yes	Open PR as draft; send approval request via Slack/email webhook; auto-merge blocked
$< 70\%$	No	Open PR as draft; send approval request via webhook with explanation
$< 70\%$	Yes	Open PR as draft; send approval request; mark as "requires security review"; 24-hour timeout before escalation

Timeout Escalation: If no human response is received within 24 hours for a critical-path approval request, the system:

1. Sends a second notification to the on-call engineer (via PagerDuty integration if configured)
2. Leaves the PR in draft state with a "stale approval" label
3. Does NOT auto-merge under any circumstances for critical-path changes

8. Technology Stack

Table 11 — Technology Stack and Design Decisions

Layer	Technology	Design Decision
Orchestration	LangGraph StateGraph + Send API + Claude Orchestrator	LLM-based Orchestrator reasons over state rather than following hardcoded edges; handles partial failures adaptively
LLM	Claude Sonnet 4.6 (<code>claude-sonnet-4-6</code>) + Claude Haiku 4.5 (<code>claude-haiku-4-5-20251001</code>)	Haiku triages/routes; Sonnet confirms bugs, plans, and generates patches; 4-retry exponential backoff with jitter
Bug detection	LLM Bug Hunter (Haiku → Sonnet) + Bandit + ESLint	Two-stage LLM hunter is the primary detector and works without external scanners; Bandit (Python) and ESLint (JS/TS) complement it where available; Bandit re-runs in the Phase 4 validation loop
Patching	Anchored SEARCH/REPLACE diff applier	Surgical minimal edits — each anchor must match the file exactly once; rejects full-file rewrites; emits a real unified diff for the PR
Dep. scanning	pip-audit + npm audit + safety	Detect CVE-flagged and outdated packages from requirements.txt / package.json; run in Phase 1 parallel
Test execution	pytest + subprocess sandbox	Validation Agent applies the patch to a temp copy and runs <code>pytest -x</code> on affected modules; <code>pytest "no tests collected"</code> (exit 5) is handled as no-tests (not a failure) and caps confidence at 60%; stdout captured for retry context
Repository Memory	ChromaDB (persistent, local)	Stores (bug_type, root_cause, fix_strategy, confidence) per resolved fix; queried by Bug Investigation via semantic similarity (cosine threshold 0.85)
Knowledge graph	Python ast + networkx	Directed graph of file-level imports, function defs, API routes, DB models; backbone of blast radius and repair ordering
Source control	PyGithub + GitHub REST API	Async branch, commit, PR; confidence score and approval status embedded in PR description
Application / UI	Streamlit	Single-page Streamlit app: sidebar config (Anthropic key, GitHub token, repo URL, branch); interactive repository bubble map with click-to-highlight blast radius (vis-network); real-time agent progress feed; findings table; repair-plan view; PR diff preview; confidence dashboard; human-approval display

Layer	Technology	Design Decision
Runtime / event transport	Background thread + in- process queue + Pydantic BaseSettings	Clicking Run launches the async pipeline on a background thread; every agent event is pushed onto a thread-safe in-process queue and rendered live in the Streamlit feed (replacing the previous FastAPI + SSE transport)
Config	Pydantic BaseSettings	Validated at startup; fails loudly before running if ANTHROPIC_API_KEY is missing; credentials supplied via sidebar inputs or environment variables / .env

9. Data Flow & Integrations

Clicking **Run Pipeline** in the Streamlit sidebar constructs a job and launches the pipeline on a background thread, returning control to the UI immediately so progress can be rendered live. Phase 1 runs Repo Mapper (networkx graph), Dependency Analyzer (pip-audit + npm audit), and the Bug Hunter — a two-stage LLM detector (Haiku triage → Sonnet confirm) plus Bandit and ESLint — concurrently. Findings are then gated: real bugs (security / functional / performance) flow on to be fixed, while low-value code-quality nits are kept as report-only and never open a PR.

Bug Investigation queries ChromaDB for similar past fixes, then reasons over the 2-hop neighbourhood from the knowledge graph to confirm the cross-file root cause and blast radius — the context that isolated file analysis cannot achieve.

Repair Planner groups the fixable findings by epicenter file and topologically sorts them, so the loop produces one pull request per file. Code Gen receives the full epicenter file plus the repair plan and returns minimal anchored SEARCH/REPLACE edits (never a whole-file rewrite); these pass through the four-gate validation loop (AST → pytest → regression → differential security). PR Author applies the confidence gate and opens one GitHub PR per file with a structured description, confidence badge, real unified diff, and blast radius summary. Throughout the run, every agent event is pushed onto a thread-safe in-process queue and streamed to the Streamlit UI in real time — providing the same live visibility the SSE feed offered, without a separate web server.

10. Cost Control & Observability

10.1 LLM Cost Optimisation

LLM calls are the dominant cost in this pipeline. The system implements a tiered model strategy:

Table 12 — LLM Tier Strategy

Task	Model	Rationale
Bug Hunter — triage (every file)	Claude Haiku	Cheap, high-recall first pass to flag candidate defects; skips clean files fast
Bug Hunter — confirmation	Claude Sonnet	Validates each candidate, drops false positives, writes evidence + a concrete fix
Orchestrator routing decisions	Claude Haiku	Structured classification task; low creativity needed
ChromaDB semantic search ranking	Claude Haiku	Structured relevance scoring
Bug Investigation	Claude Sonnet	Requires reasoning over graph + code context
Repair Planning	Claude Sonnet	Requires dependency reasoning + strategy generation
Code Generation	Claude Sonnet	Requires accurate code generation with full file context
Security Verification analysis	Claude Sonnet	Requires security expertise for differential analysis
PR Author (description generation)	Claude Haiku	Structured template fill from computed data

Token Budgeting: Each agent call has a max token limit (code gen: 8k output; investigation: 4k output; routing: 1k output). Exceeding the budget triggers a chunked approach (split file into sections) rather than unlimited spend.

10.2 Observability & Tracing

Every pipeline run produces a structured trace for debugging and auditing:

- **Structured JSON logs per agent:** Each agent emits a log entry with `agent_name`, `phase`, `input_summary`, `output_summary`, `duration_ms`, `llm_tokens_in`, `llm_tokens_out`, and `status`

(success / retry / failed).

- **Pipeline state snapshots:** The Orchestrator persists the full state (findings, repair plan, validation results) to a JSON file after each phase, enabling post-mortem analysis and resume capability.
- **Live event feed:** All agent events are pushed onto an in-process queue and rendered in the Streamlit UI in real time, providing live visibility into which agent is active, what phase is running, and what findings have been discovered.
- **Cost tracking:** Per-run cost is estimated from token counts and model prices, stored alongside the run record for budget monitoring.

10.3 Error Handling & Resilience

Table 13 — Error Handling Strategies

Failure Mode	Strategy
Claude API rate limit	Exponential backoff (1s, 2s, 4s, 8s) + jitter; max 4 retries
Claude API timeout	60-second timeout per call; on timeout, retry once then mark agent as degraded
Bandit / ESLint crash	Run tools in isolated subprocess; on crash, log error and continue with the remaining detectors
Bug Hunter LLM error on a file	Log a warning event for that file and continue; one file's failure never aborts discovery
Dependency Analyzer cannot reach CVE registry	Orchestrator skips CVE findings; continues with the remaining detectors; logs warning
Repository has no tests	Validation Agent falls back to static analysis; caps confidence at 60%; triggers human approval
Patch fails all 3 retries	Mark finding as "unresolved"; log full error trace; continue to next finding
GitHub API rate limit	Exponential backoff; if persists, queue PR for later creation

11. Expected Outcomes

On a demo repository with four planted issues (SQL injection chain, hardcoded API key, N+1 ORM query, MD5 hash usage), the pipeline completes in under three minutes, producing one PR per epicenter file. Each PR includes:

- Confidence score badge (e.g., "Confidence: 85%")
- Blast radius count (e.g., "Affects 4 modules across 2 services")
- Root-cause explanation with cross-file data-flow trace
- Human approval request where triggered (low confidence or critical path)
- Structured diff showing before/after with commentary

The Streamlit UI surfaces all of the above live: the agent feed streams each phase as it runs, and on completion the Findings, Repair Plan, and Pull Requests tabs present the results with confidence badges and approval status.

12. Scalability & Advanced Features

CI/CD Integration: A GitHub webhook triggers a run on push to main or new PR. The Validation Agent reuses the repository's existing CI environment. Configuration is a single GitHub App installation.

Knowledge Graph Persistence: The networkx graph serialises to disk (GraphML format) between runs. A second scan of the same repo skips Repo Mapper and performs incremental diff analysis (only changed files are re-parsed), reducing Phase 1 time by 80% on subsequent runs.

Parallel Validation: pytest and bandit run concurrently inside the Validation Agent; wall-clock time matches the slowest tool, not their sum.

Multi-Repository Mode (Future Work): The Orchestrator architecture supports accepting a GitHub organisation pattern and queuing one job per repository. ChromaDB memory would be shared so fixes in one repository propagate as cache hits to similar issues in others. This requires: (1) a job queue (Redis/RabbitMQ) for repo-level parallelism, (2) GitHub API rate limiting with token bucket, (3) per-repo ChromaDB collection isolation. Planned for post-hackathon development.

13. Innovation vs Existing Tools

Table 14 — Capability Comparison with Existing Tools

Capability	Semgrep AppSec	Patchwork AutoFix	This System
All 4 issue classes	Security only	Security only	Yes — functional, security, quality, performance
Real bug discovery (LLM)	Rule patterns only	Rule patterns only	Yes — two-stage Haiku→Sonnet hunter with evidence, no rule-writing
Surgical minimal-diff patches	N/A	Whole-region rewrite	Yes — anchored SEARCH/REPLACE; rejects full-file rewrites
One PR per file (grouped)	N/A	No	Yes — a file's issues fixed and shipped together
Noise control (report-only gating)	No	No	Yes — style nits never open PRs
Dependency CVE scan	Paid add-on	No	Yes — pip-audit + npm audit, Phase 1
Knowledge graph	No	No	Yes — networkx with 8 node types, 8 edge types
Cross-file root-cause	No	No	Yes — 2-hop BFS + data-flow tracing
Ordered repair planning	No	No	Yes — topological sort on affected-file graph
Real test execution	No	No	Yes — pytest + bandit in subprocess sandbox
Regression detection	No	No	Yes — blast-radius testing via knowledge graph
Differential security analysis	No	No	Yes — pre/post diff scan with new-vulnerability detection
Confidence scoring	No	No	Yes — 5-signal composite, 0–100%
Human approval gate	No	No	Yes — auto-triggered on low confidence or critical path
Repository memory	No	No	Yes — ChromaDB, shared across repos
LLM Orchestrator	No	No	Yes — adaptive, handles partial failures
Cost optimisation	N/A	N/A	Yes — Haiku/Sonnet tiering, token budgets

Capability	Semgrep AppSec	Patchwork AutoFix	This System
Observability	Limited	Limited	Yes — structured logs, per-run cost tracking, live Streamlit feed

14. Limitations & Future Work

Test Dependency: Test execution requires the target repository to have a working test suite. Without tests, the Validation Agent falls back to static analysis only, lowering the maximum achievable confidence score to 60% and triggering mandatory human approval. Future work: auto-generate characterization tests from observed API behaviour.

Dynamic Code Patterns: The AST knowledge graph handles direct imports and static decorators reliably but can miss heavily dynamic patterns (runtime module loading, extensive monkey-patching, metaclass factories). Ambiguous edges are marked with `confidence: low` and conservatively widen the blast radius. Future work: integrate symbolic execution for dynamic call-graph refinement.

Cold Start: ChromaDB memory improves with use; early runs on a fresh instance have no cache hits, resulting in lower initial confidence scores and more Code Gen retries. This is a transient issue that resolves after 5–10 successful fixes are stored.

Language Support: The current design optimises for Python (ast module, pytest, bandit) with JavaScript/TypeScript secondary support (ESLint, npm audit). Full support for Java, Go, and Rust would require additional AST parsers and security scanners.

Planned Future Work:

- Sandboxed Docker execution for full CI environment parity
- PR auto-merge for confidence $\geq 95\%$ on non-critical paths
- GPT-4o as a secondary LLM for cost optimisation on triage calls
- Multi-repository org-level scanning with shared ChromaDB memory
- 3D force-directed upgrade of the interactive bubble map (the 2D blast-radius map is already implemented — see §5.3)